NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

Official Study Material • Java Programming & OOP (JAVA101) • Unit 5: GUI Programming & Database Connectivity (JDBC)

Course Code:	JAVA101	**Regulation:**	R20 / R23 Autonomous
Subject:	Java Programming & OOP	**Unit:**	Unit 5 - GUI Programming & Database Connectivity (JDBC)

NARASARAOPETA ENGINEERING COLLEGE (AUTONOMOUS)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

ACADEMIC STUDY MATERIAL: JAVA101 JAVA PROGRAMMING & OOP

UNIT 5: GUI PROGRAMMING & DATABASE CONNECTIVITY (JDBC)

Regulation: R20 / R23 Autonomous Syllabus | Credits: 4

1. GUI PROGRAMMING WITH SWING & EVENT HANDLING:

Java Swing is a GUI toolkit providing lightweight, platform-independent GUI components built on top of AWT.

- Swing Components:

* Top-Level Containers: JFrame, JDialog, JApplet.

* Intermediate Containers: JPanel, JScrollPane.

* Atomic Controls: JButton, JLabel, JTextField, JPasswordField, JCheckBox, JRadioButton.

- Layout Managers: FlowLayout (left-to-right), BorderLayout (North, South, East, West, Center), GridLayout (rows and columns).

- Delegation Event Model:

* Event Source: Component generating event (e.g. JButton click).

* Event Object: Encapsulates event details (e.g. ActionEvent).

* Event Listener: Interface containing callback method (e.g. ActionListener with actionPerformed(ActionEvent e)).

2. JAVA DATABASE CONNECTIVITY (JDBC):

JDBC is a Java API that enables Java applications to execute SQL queries and manipulate relational databases.

- JDBC Driver Types:

* Type 1: JDBC-ODBC Bridge Driver (obsolete).

* Type 2: Native-API Driver (partly Java, requires vendor client libraries).

* Type 3: Network-Protocol Driver (all Java, connects via middleware server).

* Type 4: Thin Driver (100% pure Java, communicates directly with database socket; highly portable and industry standard).

3. JDBC CORE WORKFLOW & INTERFACES:

1. Load JDBC Driver: Class.forName("com.mysql.cj.jdbc.Driver");

2. Establish Connection: Connection con = DriverManager.getConnection(url, user, password);

3. Create Statement:

* Statement: For simple static SQL queries.

* PreparedStatement: Precompiled parameterized SQL query. Prevents SQL Injection attacks and improves performance.

* CallableStatement: Used to execute stored procedures.

4. Execute Query:

* executeQuery(): For SELECT statements, returns ResultSet.

* executeUpdate(): For INSERT, UPDATE, DELETE, DDL, returns row count int.

5. Process ResultSet: Iterate with rs.next() and rs.getString(), rs.getInt().

6. Close Connections: con.close(), ps.close(), rs.close().

4. PRACTICAL CODE EXAMPLE:

```
// Java Program demonstrating JDBC PreparedStatement CRUD operations
import java.sql.*;

public class JdbcStudentManager {
private static final String URL = "jdbc:mysql://localhost:3306/necdb";
private static final String USER = "root";
private static final String PASS = "password";

public static void insertStudent(int rollNo, String name, double gpa) {
String sql = "INSERT INTO students (roll_no, name, gpa) VALUES (?, ?, ?)";
try (Connection con = DriverManager.getConnection(URL, USER, PASS);
PreparedStatement ps = con.prepareStatement(sql)) {

ps.setInt(1, rollNo);
ps.setString(2, name);
ps.setDouble(3, gpa);

int rows = ps.executeUpdate();
System.out.println("Inserted " + rows + " student record(s) successfully.");
} catch (SQLException e) {
System.err.println("JDBC Error: " + e.getMessage());
}
}

public static void main(String[] args) {
System.out.println("Testing JDBC PreparedStatement execution.");
// insertStudent(101, "Anil Kumar", 8.8);
}
}
```

5. IMPORTANT EXAM QUESTIONS & ANSWERS:

Q1: Explain the steps involved in connecting a Java application to a database using JDBC.

Answer: 1. Load Driver Class (`Class.forName()`), 2. Create Connection (`DriverManager.getConnection()`), 3. Create PreparedStatement (`con.prepareStatement(sql)`), 4. Set parameters and execute query (`ps.executeUpdate()` / `ps.executeQuery()`), 5. Process ResultSet, 6. Close Connection in try-with-resources.

Q2: Why is PreparedStatement preferred over Statement in JDBC?

Answer: 1. PreparedStatement is precompiled by the database engine, offering faster execution when executed repeatedly with different parameters. 2. It automatically escapes input parameters, completely preventing SQL Injection security vulnerabilities.